A Novel Implementation of LiDAR Mesh Classification and Image Classifiers in Assistive Technology for the Visually Impaired

John B. Prewitt, Samarth H. Shridhar, Hrishikesh Bagalkote

Engineering and Technology

Gwinnett School of Mathematics, Science, and Technology

970 McElvaney Ln NW, Lawrenceville, GA 30044

Abstract

Current assistive technologies for the 253 million visually impaired are unsuitable for numerous reasons: unaffordability, poor ergonomic design, and computational inefficiency. Oracle is an application that efficiently classifies obstacles and relays their relative proximity using auditory and haptic feedback. Oracle utilizes the LiDAR sensor on an iPhone 12 Pro to classify objects by using convolutional neural networks to identify depth shapes. To increase classification variety, Oracle employs supplemental image classifiers, capable of classifying objects that are too geometrically complex to be identified through LiDAR depth data. To relay pertinent audio-haptic feedback to the user, Oracle implements a sectioning algorithm, dividing the camera's view into 40 subsections, sending raycasts to the center of each section. Both the sectioning algorithm and a hierarchy algorithm calculate which object is most pertinent based on distance and obstacle type. Classification accuracy, distance accuracy, and navigational time were measured to test the device's efficacy. The models (LiDAR and Image ML) had a theoretical accuracy of 99% and empirical accuracy of 97.4% in their classifications. Additionally, Oracle had an average overall absolute error of only 3.09% in its measurement of distances and yielded an average navigational time that was 28.8% lower than the White Cane across 10 obstacle courses. Statistical tests display statistical significance between Oracle's navigational time when compared to that of a White Cane, confirming the alternative hypothesis. The device can be used for affordable, non-intrusive navigational assistance, replacing technologies such as electronic canes and furthering intelligent assistive technology research.

Table of Contents

Introduction 4

Hypotheses 5

Variables 6

Materials & Software 6

Procedure 7

Part A: Development Process (Full Codebase) 7

Part B: Data Collection and Training (LiDAR Classifiers) 9

Part C: Data Collection and Training (Image Classifiers) 10

Part D: Testing Traversal Times 10

Part E: Testing Empirical Classification Accuracy 11

Part F: Testing Empirical Distance Accuracy 12

Data 13

Figure 1 13

Tables 1-4 13

Discussion 14

Conclusion 16

Literature Cited 18

Appendix 20

Acknowledgements 20

Introduction

Although assistive technologies for the visually impaired come in staggering numbers, cost, intrusiveness, and inflexibility hinder the possibility of popularizing assistive technologies beyond the white cane (Zhao et al., 2018). For example, a study on the efficacy of ultrasonic white canes illustrated no major impact of their usage, despite the device's ability to detect nearly 70% of surrounding objects accurately (dos Santos et al., 2020). Combined with a trend of insignificant results, the monetization of assistive technologies likewise prevents people from using these devices. With visual therapy costing the average American between 1,000 and 7,000 dollars, affordability must come as a priority in supplying visually impaired individuals with assistive technology ("*Vision Therapy,"* n.d.). There are 253 million people with visual impairments around the world with 15 million of them also being deaf (Tapu et al., 2018), creating a stark need for visual assistive technology and an absence of devices to meet these demands.

Image classifiers encapsulate a domain of predictive modeling in which machine learning algorithms identify patterns among a dataset of images to produce categorical predictions. These algorithms are useful in developing assistive technology because, unlike the relatively linear output of sensors, image classifiers yield categorized inputted data into familiar terms. For example, objects surrounding a visually impaired individual can be classified under a label, such as "table" or "wall." However, current research merging assistive technology with image classifiers is often intrusive because of the technology needed to support the exorbitant computational load. The National Institute of Health lists the social barriers of assistive technology as a primary reason for the physically challenged avoiding assistive technology

("U.S. Department," n.d.). Hence, efforts must be made to develop non-intrusive assistive technologies.

Reporting the distance of an object relative to a user's position is a prime topic for current assistive technology research. The technology of choice for collecting depth data is RGB-D cameras, which unfortunately come with considerable costs. The cheap substitutions for RGB-D cameras are few and cannot report distance to a similar caliber of accuracy (Shimakawa et al., 2017). The solution lies in mobile devices. Going forth, mobile device manufacturers are incorporating LiDAR technology into the device cameras. By leveraging this trend, reporting accurate distances and object classifications can seamlessly marry in a non-intrusive, cost-free format: the mobile app (Kowalczuk & Szymański, 2019). This paper details the development and testing of a mobile application, Oracle, attempting to solve the unaffordability, computational inefficiency, and discomfort of current assistive technologies for the visually impaired.

Hypotheses

Null: If a visually impaired person utilizes the mobile application, there is no statistically significant difference in the navigational time through the course than a case in which the device was not utilized.

Alternative: If a visually impaired person utilizes the mobile application, the visually impaired persons will experience a decrease in navigational time through their environment compared to usage without the system and with traditional navigation methods (i.e. white cane).

Variables

The research is subdivided into two stages, classification and experimentation. The classification phase gauges the performance of the image classifiers from the accuracy of its classifications. The experimentation stage tests the efficiency of utilizing the mobile application by examining the walking speeds of a visually impaired test subject across various trials. The independent variable of the classification phase will be the types of objects that the mobile application classifies. The possible classifications for the model are as follows: wall, floor, seat, table, window, ceiling, toilet, box, garage, stairs, trees, desk, and fences. The independent variable for the experimentation phase is the type of device used to aid or hinder the user's navigational sense as they traverse a controlled environment. The types of navigation include no aid (normal vision), blindfold (to ensure visual impairment), a white cane, and the mobile application, Oracle. No aid and blindfold serve as controls. The dependent variable of the classification phase is the theoretical and empirical accuracies outputted by the model. The dependent variable of the experimentation phase is the time required to traverse throughout the environment when paired with a specific condition. The values will be the amount of time it takes to fully complete a controlled course, measured in seconds. The controlled variables include the phone model, code, the blindfold, the iPhone holster, the white cane, and the individual participating in the experimentation.

Materials & Software

- Apple iPhone 12 Pro
- iPhone Holster (Otterbox Defender Series)
- White Cane
- XCode Development Studio

Procedure

Part A: Development Process (Full Codebase)

- **Initializing the ARKit on iPhone 12 Pro:** Using structure from the Apple Scene Semantics sample code, create a series of event handlers that will connect the LiDAR scanner to the codebase and allow the depth and image frames to be utilized and manipulated by the future algorithms.
- **Retrieving Depth and Color Frames:** Inside of these event handlers perform a bit cast of the depth map and color maps onto a CVPixelBuffer that will place the contents of each frame into an easily analyzable and manipulatable state. Once transferred, remove the depth and color frames from memory for a less battery-intensive process.
- **Intuitive Graphical User Interface (GUI):** Inside the XCode's integrated development environment, create a new storyboard application. Inside the storyboard application for settings, create a slider to increase or decrease the time interval between callouts to the users preference, a picker view to select the language of the callout and distance, and two buttons for critical options for the user: use of the Imperial or Metric systems for distance callouts and whether or not haptic vibrations should be used (Appendix D). Furthermore, on the main storyboard application, the ARView should be displayed alongside a mesh created by the LiDAR scanner for easier debugging as well as navigation to the settings menu. All of the menus must be adapted to meet the requirements for Apple's text-to-speech screen readers for maximum accessibility for the visually impaired.
- **LiDAR Classifications:** Once a raycast is sent from the LiDAR sensor on the iPhone, the distance of the obstacle at the point where the raycast is sent as well as the type of obstacle at this point is recorded. The distance is measured using the time it takes for the ray of light

to leave and return to the sensor after reflecting off the object and calculated with this time and the known speed of light. The type of obstacle is determined using the distances from raycasts surrounding our specified raycast and analyzing their relations with one another (For example, a raycast with a large area of similar distance raycasts surrounding it would likely be a wall.)

- **Sampling and Sectioning of LiDAR Mesh**: Create a function that subdivides the LiDAR Mesh into *n* rectangular sections (See Appendix C). The function must then dynamically render points at the middle of each rectangle in the mesh. Subsequently, a raycast must be sent at each of these points to obtain a LiDAR classification and the distance data. These classifications are then stored in a structure class for access during the hierarchy phase.
- **Supplemental Image Classifiers (Convolutional Neural Network):** For objects that are too geometrically complex to be classified via LiDAR classification, train a convolutional neural network on common obstacles (cars, people, beds, etc.) to supplement image classification. The supplemental image classifiers enable the application to alert the user of a greater variety of obstacle types and relay auditory feedback in situations with low LiDAR model confidence.
- **Hierarchy:** The Hierarchy algorithm is used to determine which callout to make to the user using the speech-to-text method. The algorithm is called once every three seconds and determines which identified obstacle in the user's frame is most pertinent to the user. This algorithm takes three factors into account: depth, classification, and the proximity to center. The Hierarchy uses these three factors to rank each of the sampled points of the sampling algorithm. The highest ranked point will then be analyzed alongside the supplemental Convolutional Neural Network (CNN) before sending auditory and haptic feedback to the

user. If the CNN returns an image label with 95 percent or greater confidence and is also labeled as a precedence category (i.e. people), then the image classification will be sent to the audio method. If not, then the LiDAR classification will be relayed.

- **Haptic Feedback Translation:** Once the distances are obtained from the LiDAR scanner, they are run through a switch statement that determines if the obstacle is within close proximity and if so the magnitude of the haptic feedback. The closer the object is to the LiDAR sensor, the higher magnitude and longer duration of the vibration of the phone.
- **Relaying Audio Feedback for Explicit Classification and Distance Data:** Using the distance and classification determined by the Hierarchy algorithm, a text-to-speech command will be used to relay the information auditorily to the user. Furthermore, using the settings UI, these callouts can be personalized to the liking of the user (see Appendix D). The user has the option of changing the callout language to English, Spanish, French, Mandarin, Hindi, or Arabic. The user also has the option of selecting imperial (inches) or metric systems (centimeters) for the distances. Finally, the user has the option to alter the default time between callouts (three seconds). Once the user selects their preferences, the application makes a callout using the speech synthesizer.

Part B: Data Collection and Training (LiDAR Classifiers)

1. Initialize the LiDAR camera and enable the mesh.
2. Retrieve the Depth frames and convert them into a CVPixelBuffer to access depth values
3. Dynamically section the mesh into *n* squares and send a raycast into the center of each individual square in the mesh.
4. Using the distance data obtained from the raycast, select the data cluster closest to the LiDAR sensor.

5. Categorize the shape of the distance data by object and compile into a training dataset.
6. Partition training and testing depth data. If the accuracy is hindered by factors such as overfitting or logarithmic loss, revise the model as necessary.

Part C: Data Collection and Training (Image Classifiers)

1. Access the Google Open Images Repository.
2. Filter through the repository and compile image data categorically.
3. Partition data into training and testing subsections.
4. Write a Python script to artificially distort, reorient, and blur images randomly to resemble real use-cases in which the mobile device is in motion.
5. Train the model for twenty-five iterations using convolutional neural networks in Apple's CreateML studio. Evaluate the model's accuracy. If the accuracy is hindered by factors such as overfitting or logarithmic loss, revise the model as necessary.

Part D: Testing Traversal Times

1. Download and compile all the code
2. Secure the iPhone in the case holster and secure the clip on the user's waist to enable the app. View Appendix G for a photo of the device secured in the holster and Appendix B for a photo of a user with the holster.
3. Prepare ten obstacle courses to test the traversal time of the device. Focus course modeling on replicating daily interactions with common household objects and staircases. View the CAD models in Appendix E and Appendix F for example course models.
4. Render a table with the three traversal methods that require a blindfold (Oracle, blindfold, and white cane) with ten courses. Assign each element in this table a number from one to thirty. Randomly generate numbers between one and thirty that represent each element,

and the order each element appears in will be the order testing occurs. There must be no repetition of the same element. Please find below details describing the traversal methods in this table:

a. 1st Scenario (Negative Control / Blindfold): Test the time to complete each course with full blindness but no visual aids. This is a control that tests how difficult it is for a blind person to travel through each course.
b. 2nd Scenario (Blindfolded with White Cane): Test the time to complete each course with a white cane and a blindfold.
c. 3rd Scenario (Blindfolded with Oracle Application): Test the time to complete each course with the Oracle Application and a blindfold.

5. Create a new table using the same randomization system. However, this time, the only traversal method will be corrected vision without a blindfold. Instead of numbering the elements one to thirty, number them one to ten.
    a. 1st Scenario (Positive Control / Normal Vision without Blindfold): Test the time to complete each course with clear, corrected vision without any visual a ids. This control tests how quickly one can casually travel throughout each course.
6. Record the time to complete each trial in the order they were randomly generated.

See Appendix H for a sample trial of the Oracle Application on Course 1 (Appendix E)

Part E: Testing Empirical Classification Accuracy

1. Download and compile all the code.
2. Place Oracle in front of an object from the classification library.

3. Allow Oracle to call out a classification for the object fifty times. Record each time the callout is incorrect. Alter the position and angle of Oracle around the object for each classification.
4. Repeat steps two and three for every object in the classification library.
5. Calculate and record the percent accuracy of Oracle's performance on each object based on the number of incorrect classifications for that object.

Part F: Testing Empirical Distance Accuracy

1. Download and compile all the code.
2. Place Oracle 20 centimeters from a discrete object (a wall, for example).
3. Enable Oracle to call out its distance from the object. If the distance is incorrect, record the percent error of the value Oracle called out to the known distance value (for the first trial, 20 centimeters).
4. Allow Oracle to call out the distance from the object five times. Each time, record the percent error. Finally, average the percent error of each instance and record the average absolute percent error for that distance.
5. Repeat steps three and four for distances from 20 centimeters to 480 centimeters in increments of 20.
6. Average the average absolute percent error from all the trials. Record this value as the overall absolute percent error.

*See Appendix A for a flowchart of the procedure

Data

Figure 1: Timing of Each Traversal Method per Course

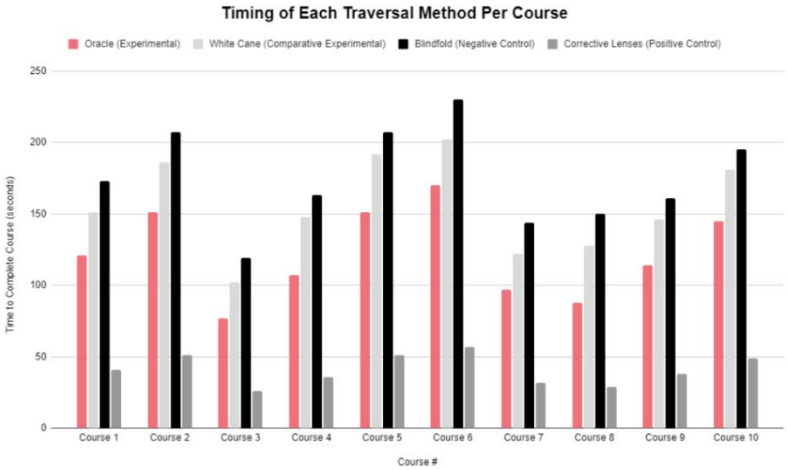


Table 1: Distance Callout Error

Actual Distance (cm)	Average Absolute Percent Error (%)
20	1.22
40	1.25
60	1.41
80	1.24
100	1.85
120	2.24
140	2.59
160	1.98
180	2.47
200	2.51
220	3.02
240	3.09
260	2.84
280	3.05
300	3.12
320	4.21
340	3.97
360	4.33
380	3.79
400	4.53
420	4.58
440	4.89
460	4.65
480	5.25
Overall Absolute Percent Error (%)	***3.09%***

Table 2: Kruskal-Wallis H Test (Oracle, White Cane, Blindfold)

K (Observed value)	8.776
K (Critical value)	5.991
DF	2
p-value (one-tailed)	0.012
alpha	0.050

Table 3: Pairwise Mann-Whitey Test (Oracle vs White Cane)

U	23
U (standardized)	0.000
Expected value	50.000
Variance (U)	174.474
p-value (Two-tailed)	0.043
alpha	0.050

Table 4: Real Obstacle Faced vs. LiDAR Classification of Obstacle

		Real Obstacle Faced						
		Table	*Floor*	*Ceiling*	*Door*	*Seat*	*Window*	*Wall*
Obstacle Detected by LiDAR Classification Model	*Table*	98%	0%	0%	0%	2%	0%	0%
	Floor	0%	100%	0%	0%	0%	0%	0%
	Ceiling	0%	0%	100%	0%	0%	0%	0%
	Door	0%	0%	0%	96%	0%	0%	2%
	Seat	2%	0%	0%	0%	98%	0%	0%
	Window	0%	0%	0%	0%	0%	100%	0%
	Wall	0%	0%	0%	4%	0%	0%	98%

Table 5: Real Obstacle Faced vs. Image ML Classification of Obstacle

		Real Obstacle Faced						
		Box	*Cars*	*Desk*	*Fences*	*People*	*Stairs*	*Toilet*
Obstacle Detected by Image ML	*Box*	96%	0%	4%	6%	0%	0%	4%
	Cars	0%	100%	0%	0%	2%	0%	0%
	Desk	4%	0%	96%	2%	0%	0%	0%
	Fences	0%	0%	0%	92%	0%	0%	0%
	People	0%	0%	0%	0%	98%	4%	0%
	Stairs	0%	0%	0%	0%	0%	96%	0%
	Toilet	0%	0%	0%	0%	0%	0%	96%

Discussion

The overarching trend observed was that utilizing the Oracle Application was consistently faster than using a white cane, as portrayed in Figure 1. To confirm statistical significance, a Kruskal-Wallis test was run to evaluate variance overall visual aid methods, and a Mann-Whitney U test was run to evaluate pairwise variance. The independent variable is the visual aid method with the dependent variable being course traversal time.

Because of the non-uniform distribution of the data, we selected non-parametric alternatives to ANOVA and Student's T-tests, the Kruskal-Wallis Test (Figure 3), and Pairwise Mann Whitney U Tests (Figure 4), respectively. The α-value for both tests was set to 0.05 so that we could achieve a 95% confidence interval. The Kruskal-Wallis test depicted in Table 2 reported a p-calculated value of 0.012. The Mann Whitney U test depicted in Table 3 reported a p-calculated value of 0.043 between white cane and Oracle. The positive control case, corrected vision, was omitted from the tests because of the inherent statistical significance it injects into each test. Every p-calculated value pertaining to pairs with the corrected vision case was <0.05. Therefore, the test case would have skewed the Kruskal-Wallis test results because of its significant difference from the other test cases. Generally, the blindfold cases reported the slowest times among the four traversal methods. The quickest traversal method (disregarding normal vision) was achieved utilizing the Oracle application.

Since the Kruskal-Wallis test's p-value, 0.012, is less than the α-value of 0.05, the Kruskal-Wallis test confirms statistical significance between using the white cane, Oracle, and the blindfold. However, the Kruskal-Wallis test fails to specifically identify whether Oracle is a significant contributor to statistical significance. Thus, the pairwise Mann Whitney U test identifies intergroup correlations to pinpoint sources of possible significance. The Mann-

Whitney U test between Oracle and the white cane yielded a p-value of 0.043, smaller than the α-value of 0.05 and therefore statistically significant. Hence, this result supports the rejection of the null hypothesis and the claim that using Oracle is significantly faster than using a white cane or no assistive technology while traversing the course.

We speculate that utilizing Oracle results in faster navigation because Oracle combines two distinct aids, audio and haptic feedback, to facilitate movement. As depicted in figures five and six, Oracle classifies objects at empirical accuracies consistently exceeding 90%. As illustrated in figure two, Oracle outputs distances with a minimal average absolute percent error of 3.09%. Hence, Oracle combines accurate classifications and distance callouts with versatile audio-haptic feedback. Additionally, Oracle significantly outperforms white canes in the identification of objects such as staircases. This combination of aids and mechanistic advantages helps users more efficiently identify obstacles, locate them, and avoid them with Oracle.

There were possible sources of error in the experimentation phase that require acknowledgment. The first source of error resulted from having a single test subject. A single test subject limits the testing age range and renders a small sample size. Additionally, the test subject was legally blind, but only without corrective lenses. Thus, the test subject was not thoroughly accustomed to living life without vision. Nonetheless, the subject's blindness without corrective lenses yielded a much stronger testing candidate than a test subject with zero visual impairments. According to a comparative study with blind and ordinary vision people, subjects with uncorrected visual impairments tended to yield more accurate, consistent results (dos Santos et al., 2020). Thus, the test subject's designation as a legally blind individual without corrective lenses still renders him a suitable test subject.

The first significant method our group would fundamentally change is utilizing instance segmentation in the training data for the supplemental image classifiers. The images inputted as training data for the supplemental image classifiers occasionally had several distinct objects in the background of the image that did not correspond to the image's classification label. An instance segmentation algorithm would outline and label each object in the image, thus minimizing the possibility of misclassification errors.

Conclusion

Oracle's speed and efficiency are proven through the analyses of the data as the application's traversal time was significantly faster than the competing traversal methods (white cane, blindfold) as the Kruskal-Wallis Test value had a p-calculated value less than 0.05, accepting our alternative hypothesis. Additionally, Oracle's accuracy in classification and distance recognition for objects is unparalleled in comparison to other visual assistive technologies, exhibiting a 99% theoretical classification industry and a 97.4% empirical classification accuracy alongside an average absolute error of ±3.09% for distances determined from 700 classification test cases and 120 distance test cases. As such, Oracle is an effective and affordable alternative to numerous assistive technologies and offers functions that rival and triumph over similar technologies. Oracle has accomplished elevating a mass-produced sensor to pioneer the actively researched field of combining machine learning and assistive technology, and thus the device has practical applications for affordable visual therapy that can extend into parts of the world without sustainable access to dedicated expensive equipment. Other features of Oracle such as language selection, scalability, haptic feedback, and data validation further improve the effectiveness of Oracle in these underserved demographics. Oracle, a free

application, provides pioneering software that can process, classify, and communicate the proximity of many obstacles with ample computational efficiency and time complexity. Compared to the praised assistive technology, The Sound of Vision, Oracle identifies a substantial range of potential obstacles while also classifying them—a function that The Sound of Vision lacks (Hoffman et al., 2018). Additionally, Oracle is a scalable and dynamic application, being able to accommodate more classifications to meet the demands of the visually impaired. Other popular technologies such as TapTapSee and Drishti Navigation utilize GPS for obstacle identification and are subsequently restricted to GPS-distinguishable zones. However, Oracle functions in a significantly broader range of environments—with minimal requisite APIs, offline-use, supports ten languages, and testing performed on objects that will appear in realistic, everyday scenarios. The ultrasonic white cane, a comparably versatile device, does not detect obstacles to the accuracy of the Oracle with only 72% accuracy (dos Santos et al., 2020) and is not statistically faster than the white cane, the primary device it is attempting to exalt.

As LiDAR sensors become commonplace in smartphones across the globe, Oracle's user base of visually impaired people will only magnify. While the number of platforms expands the Oracle application can continue to grow and experience improvements to ensure a comprehensive app for the visually impaired. The most notable change could be the creation of a publicly accessible database for users around the world to collect their own data through the app, contributing to Oracle's library of classifications. Furthermore, expanding upon our current haptic feedback to provide full distance and classification information to deafblind people through a braille board is a critical step in creating a polished experience for all. Lastly, incorporating additional common obstacles into the LiDAR classification model for more optimized classifications is a key step in forming a comprehensive application for the visually

impaired. With a score of features, classifications, and increased affordability compared to traditional assistive devices, Oracle can be instrumental in facilitating universal, nonintrusive visual therapy to the visually impaired, furthering machine learning and rehabilitative technology research, and reprioritizing assistive technology industry on affordability and reliability.

Literature Cited

Classifying Images with Vision and Core ML. Apple Developer Documentation. (2020). Retrieved October 15, 2021, from https://developer.apple.com/documentation/vision/classifying_images_with_vision_and_core_ml.

Core ML models. Models - Machine Learning - Apple Developer. (n.d.). Retrieved January 5, 2022, from https://developer.apple.com/machine-learning/models/

dos Santos, A. D., Medola, F. O., Cinelli, M. J., Garcia Ramirez, A. R., & Sandnes, F. E. (2020). Are electronic white canes better than traditional canes? A comparative study with blind and blindfolded participants. *Universal Access in the Information Society*, *20*(1), 93–103. https://doi.org/10.1007/s10209-020-00712-z

Hoffmann, R., Spagnol, S., Kristjánsson, Á., & Unnthorsson, R. (2018). Evaluation of an Audio-haptic Sensory Substitution Device for Enhancing Spatial Awareness for the Visually Impaired. Optometry and vision science: official publication of the American Academy of Optometry, 95(9), 757–765. https://doi.org/10.1097/OPX.0000000000001284

Kowalczuk, Z., & Szymański, K. (2019, September 10). Classification of objects in the LIDAR point clouds using Deep Neural Networks based on the PointNet model.

Retrieved October 19, 2021, from https://www.sciencedirect.com/science/article/pii/S2405896319304331

MLImageClassifier. Apple Developer Documentation. (2020). Retrieved October 15, 2021, from https://developer.apple.com/documentation/createml/mlimageclassifier.

Shimakawa, M., Akutagawa, R., Kiyota, K., & Nakano, M. (2017). A Study on Staircase Detection for Visually Impaired Person by Machine Learning using RGB-D Images. Proceedings of The 5th IIAE International Conference on Industrial Application Engineering 2017. doi:10.12792/icisip2017.080

Tapu, R., Mocanu, B., & Zaharia, T. (2018). Wearable assistive devices for visually impaired: A state of the art survey. Pattern Recognition Letters, 137, 37–52. https://doi.org/10.1016/j.patrec.2018.10.031

U.S. Department of Health and Human Services. (n.d.). *Rehabilitative and Assistive Technology*. Eunice Kennedy Shriver National Institute of Child Health and Human Development. Retrieved January 2, 2022, from https://www.nichd.nih.gov/health/topics/rehabtech

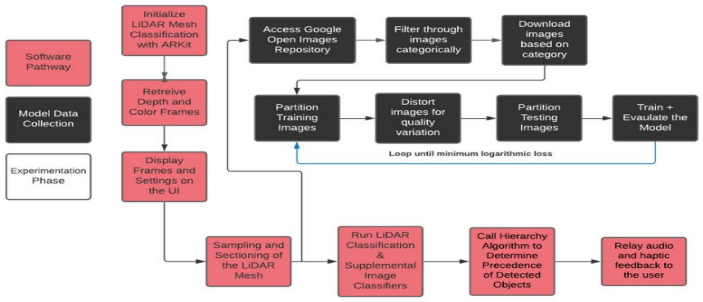
Vision therapy costs: The vision therapy center. Vision Therapy Costs | The Vision Therapy Center. (n.d.). Retrieved January 2, 2022, from https://www.thevisiontherapycenter.com/costs-the-vision-therapy-center

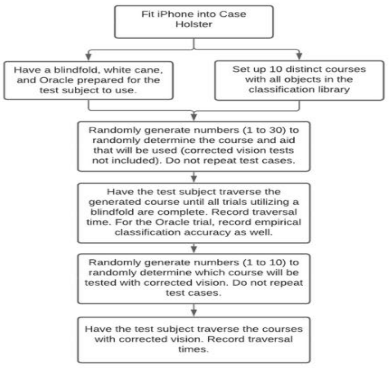
Visualizing and Interacting with a Reconstructed Scene. (n.d.). Retrieved January 5, 2022, from https://developer.apple.com/documentation/arkit/content_anchors/visualizing_and_interacting_with_a_reconstructed_scene

Zhao, Yuhang & Kupferstein, Elizabeth & Tal, Doron & Azenkot, Shiri. (2018). "It Looks Beautiful but Scary": How Low Vision People Navigate Stairs and Other Surface Level Changes. 307-320. doi:10.1145/3234695.3236359

Appendix

Appendix A

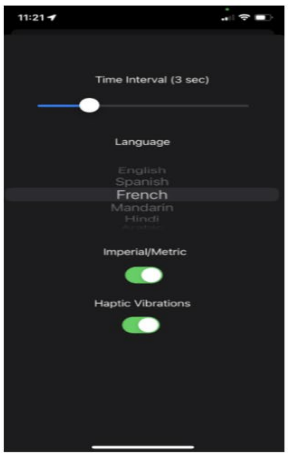




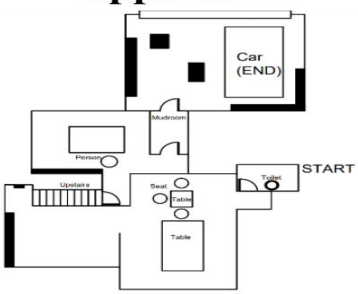
Appendix B

Appendix C

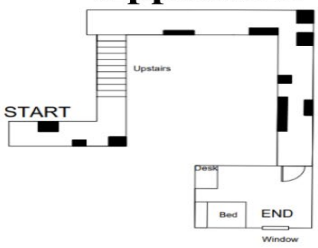
Appendix D



Appendix E



Appendix F



Appendix G

Appendix H: Link to Demonstration Trial:

https://drive.google.com/file/d/1UDUjAMYN9o3ukPZ7a5AU71Pw-KHSN6sM/

Acknowledgements

We would like to acknowledge Ms. Berry, who advised us to ensure our success, Mr. Prewitt, who equipped us with necessary materials, and Dhruv Gowda, for serving as a test subject.